# Project - High Level Design

# on

# Haven AI

Course Name: Gen AI

***Institution Name:*** Medicaps University – Datagami Skill Based Course

| Sr no | Student Name | Enrolment Number |
|---|---|---|
| 1. | Raj Kumawat | EN22CS301779 |
| 2. | Rajvardhan Singh Rathore | EN22CS301786 |
| 3. | Ronak Bhawsar | EN22CS301824 |
| 4. | Rudra Sthapak | EN22CS301828 |
| 5. | Rupendra Singh Tomar | EN22CS301833 |

*Group Name: 04D7*

*Project Number: GAI-27*

*Industry Mentor Name: Aashruti Shah*

*University Mentor Name: Dr.Hemlata Patel*

*Academic Year: 2025-2026*

# Table of Contents


| | | |
|---|---|---|
| Chapter 1 | Introduction | 3 |
| | 1.1. Scope of the Document | 3 |
| | 1.2. Intended Audience | 3 |
| | 1.3. System Overview | 3 |
| Chapter 2 | System Design | 4-7 |
| | 2.1. Application Design | 4 |
| | 2.2. Process Flow | 4 |
| | 2.3. Information Flow | 5 |
| | 2.4. Components Design | 6 |
| | 2.5. Key Design Considerations | 6 |
| | 2.6. API Catalogue | 7 |
| Chapter 3 | Data Design | 8-9 |
| | 3.1. Data Model | 8 |
| | 3.2. Data Access Mechanism | 8 |
| | 3.3. Data Retention Policies | 9 |
| | 3.4. Data Migration | 9 |
| Chapter 4 | Interfaces | 10 |
| Chapter 5 | State and Session Management | 11 |
| Chapter 6 | Caching | 12 |
| Chapter 7 | Non-Functional Requirements | 13 |
| | 7.1. Security Aspects | 13 |
| | 7.2. Performance Aspects | 13 |
| Chapter 8 | References | 14 |

# 1. Introduction

## 1.1. Scope of the Document

This document provides a High-Level Design (HLD) of the Real Estate Quiz/Tutor Bot, an AI-driven educational platform developed using Generative AI technologies. The scope of this document is to present an overview of the system architecture, major components, workflows, and integration of technologies such as Large Language Models (LLMs) and local storage systems.

It highlights how the system is structured at a macro level without going into implementation details, focusing on how different modules interact to deliver an intelligent and interactive learning experience. The document also outlines system boundaries, major functionalities, and high-level workflows required for development and deployment.

## 1.2. Intended Audience

This document is intended for project stakeholders, system architects, evaluators, and development teams who require a conceptual understanding of the system design. It helps in understanding how Generative AI is integrated into an educational real estate system and how different components collaborate to achieve the desired functionality.

It is also useful for academic evaluation, as it provides a clear overview of the system's architecture, design decisions, and technological stack in a simplified and structured manner.

## 1.3. System Overview

The Real Estate Quiz/Tutor Bot is an AI-powered application designed to enhance user knowledge in the real estate domain through interactive quizzes and personalized feedback. The system uses Generative AI to dynamically generate questions based on real estate concepts such as property listings, pricing, and location analysis.

Users interact with the system through a quiz interface, where they are presented with questions generated by an LLM. The system evaluates user responses using semantic understanding and provides detailed explanations for incorrect answers, creating a continuous learning loop.

The backend system handles API requests and communicates with the Generative AI module, while local storage is used to store quiz data, user responses, and real estate information. The overall system is designed to be scalable, efficient, and user-friendly.

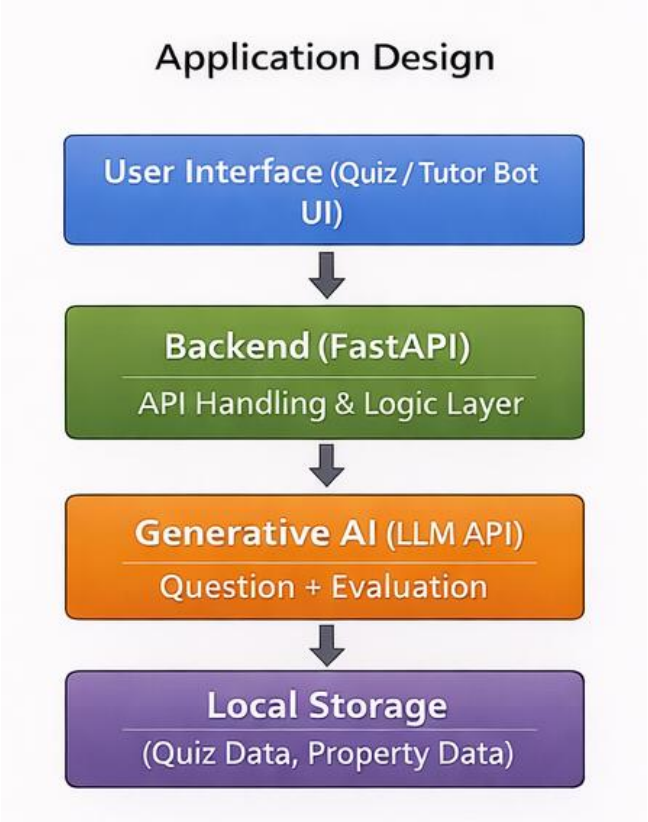
# 2. System Design

## 2.1. Application Design

At a high level, the application follows a layered architecture consisting of the user interface, backend services, Generative AI module, and local storage. The user interface serves as the interaction point where users participate in quizzes and receive feedback.

The backend layer acts as the central controller that manages user requests, processes data, and coordinates with the AI module. The Generative AI layer uses LLM APIs to generate quiz questions and evaluate user responses. Local storage is used to store domain-specific data such as property listings and quiz-related information.

This architecture ensures separation of concerns, allowing each layer to function independently while maintaining seamless communication with other components.

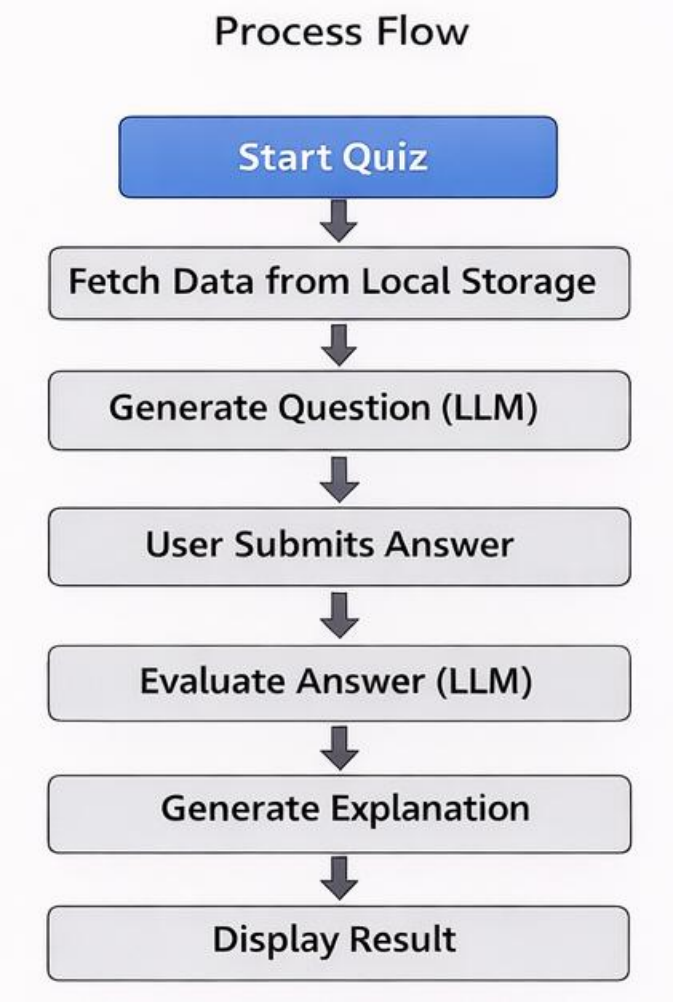



## 2.2. Process Flow

The system operates through a structured workflow that ensures smooth interaction between the user and the AI system. The process begins when the user initiates a quiz session. The backend retrieves relevant real estate data from local storage and sends it to the Generative AI module.

The AI module generates a quiz question, which is presented to the user. After the user submits an answer, the backend forwards it to the AI module for evaluation. The AI analyzes the response, assigns a score, and generates a contextual explanation. The result is then displayed to the user, completing the interaction cycle.

This process is repeated for multiple questions, creating an engaging and interactive learning experience.

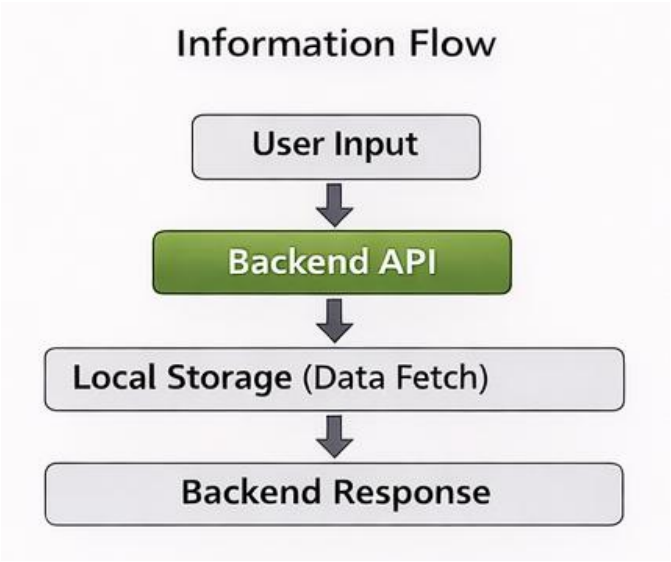



## 2.3. Information Flow

The information flow describes how data moves across different components of the system. User input is first received by the backend API, which processes the request and retrieves necessary data from local storage.

The retrieved data is passed to the Generative AI module, which performs question generation or answer evaluation. The output generated by the AI, including scores and explanations, is sent back to the backend. The backend formats this information and delivers it to the user interface.

This continuous flow ensures real-time communication and efficient processing of user interactions.

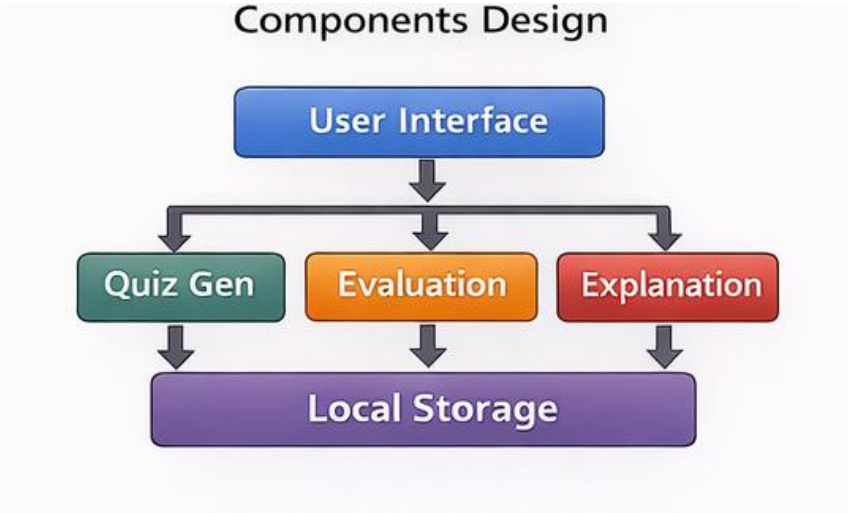



## 2.4. Components Design

The system consists of several high-level components that work together to deliver functionality. The User Interface component handles user interaction and displays quiz content. The Backend API component manages system logic and communication between modules.

The Generative AI component is responsible for generating questions, evaluating answers, and producing explanations using LLMs. The Local Storage component stores real estate data, quiz questions, and user responses.

Each component is designed to perform a specific function while maintaining integration with other components, ensuring a modular and scalable system architecture.

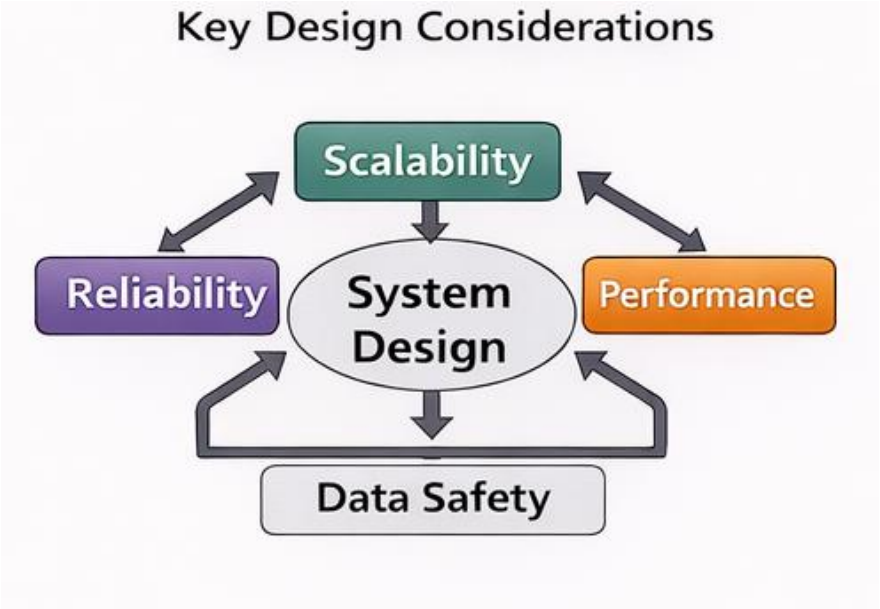


## 2.5. Key Design Considerations

The design of the system focuses on simplicity, scalability, and performance. One of the key considerations is ensuring that the Generative AI module produces accurate and meaningful content. The system must handle user interactions efficiently while maintaining low response time.

Scalability is also important, as the system should be able to support multiple users simultaneously. The use of local storage ensures quick data access and reduces dependency on external systems. Additionally, the architecture is designed to be flexible, allowing future enhancements such as cloud deployment or integration with advanced AI models.

Security and reliability are also considered to ensure safe handling of user data and consistent system performance.




## 2.6. API Catalogue

The system exposes a set of high-level APIs to manage user interactions and system operations. These APIs include endpoints for starting a quiz, retrieving questions, submitting answers, and receiving feedback.

The APIs act as the communication bridge between the frontend and backend, ensuring smooth data exchange. Each API follows a standardized structure and is designed to handle requests efficiently while maintaining scalability and reliability.

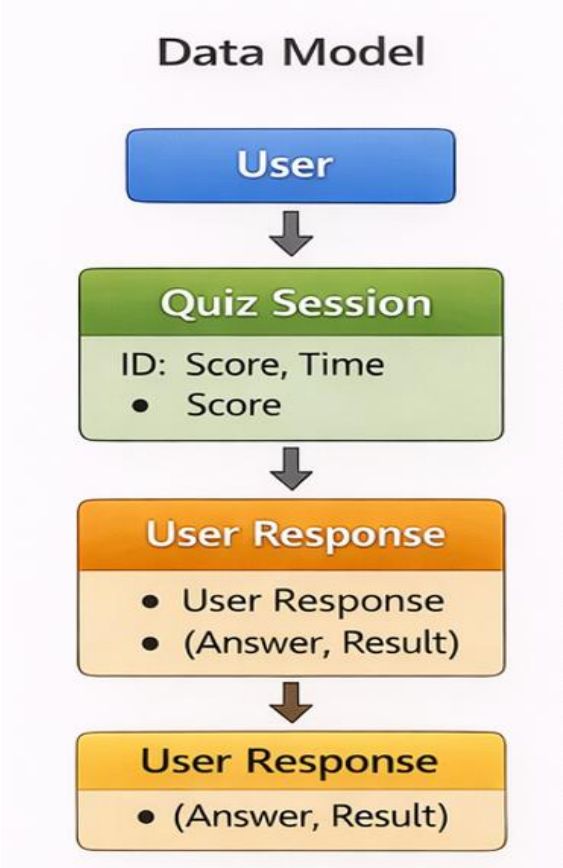
# 3. Data Design

## 3.1. Data Model

The data model of the Real Estate Quiz/Tutor Bot is designed to efficiently store and manage information related to real estate concepts, quiz questions, and user interactions. The system uses structured local storage to maintain different categories of data, ensuring quick access and easy management.

The primary data entities include user data, quiz sessions, questions, and responses. Each quiz session is associated with a user and contains multiple questions along with the user's answers and scores. The question dataset includes dynamically generated questions along with metadata such as topic, difficulty level, and correct answer references. Real estate data, such as property descriptions, pricing details, and location-based insights, is also stored and used as a knowledge base for generating quiz content.

The relationships between these entities are maintained logically, allowing the system to track user progress and generate personalized feedback. The data model is flexible and can be extended in the future to include additional attributes such as user preferences or performance analytics.

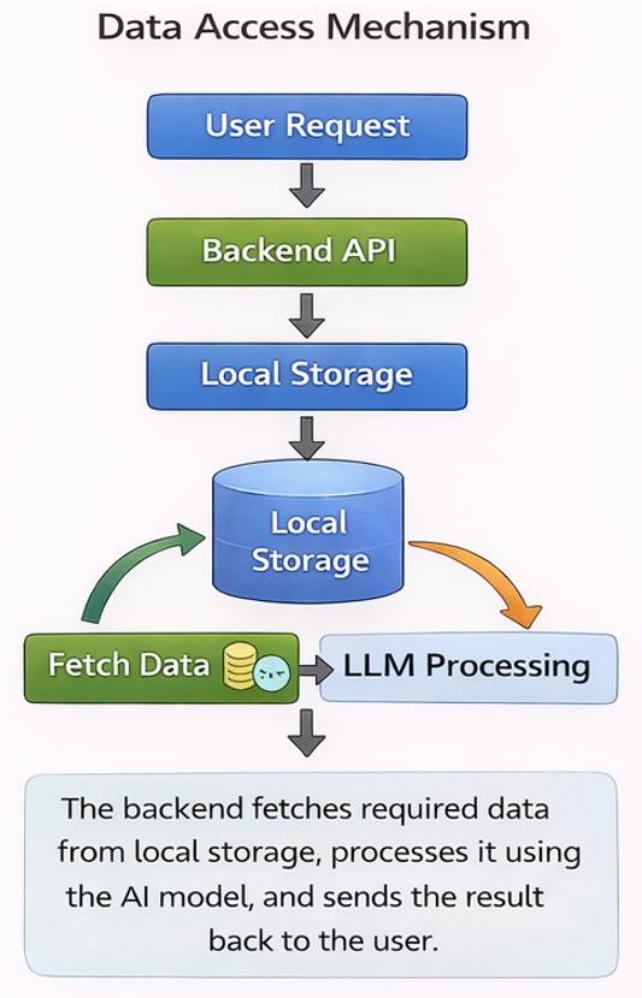


## 3.2. Data Access Mechanism

The system uses an efficient data access mechanism to retrieve and manage information stored in local storage. The backend, implemented using Python, interacts with the storage layer through well-defined functions and queries that ensure fast and reliable data retrieval.

When a user initiates a quiz, the backend fetches relevant real estate data and passes it to the Generative AI module for question generation. Similarly, when a user submits an answer, the system retrieves the associated question and context to evaluate the response accurately.

To optimize performance, the system minimizes redundant data access by storing frequently used information temporarily in memory. The data access mechanism is designed to ensure consistency, accuracy, and minimal latency, enabling smooth interaction between the backend and the AI module.

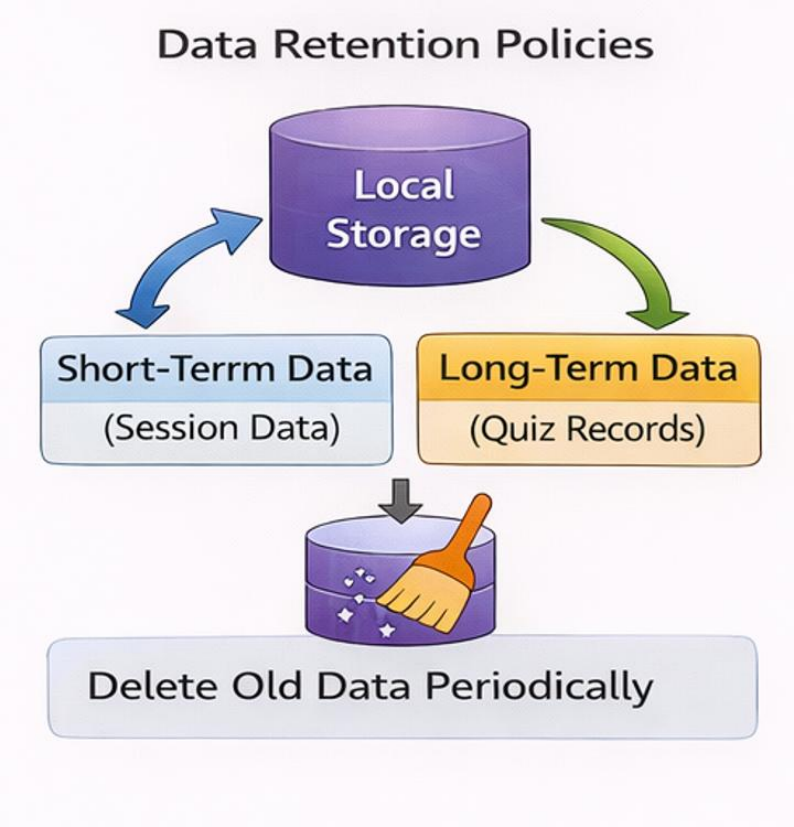



## 3.3. Data Retention Policies

Data retention policies are implemented to manage storage efficiently and ensure that the system remains scalable over time. The system retains essential data such as user quiz history, scores, and performance metrics for a defined period to support learning analysis and feedback generation.

Temporary data, such as intermediate AI responses and session-specific information, is stored only for the duration of the session and is cleared once the session ends. This helps in reducing storage overhead and maintaining system performance.

Additionally, periodic cleanup mechanisms can be implemented to remove outdated or unused data, ensuring that the storage remains optimized. These policies help balance data availability with system efficiency and scalability.




## 3.4. Data Migration

The system is designed with flexibility to support future data migration requirements. Although the current implementation uses local storage, the architecture allows seamless migration to more advanced storage solutions such as cloud databases or distributed systems.

Data migration can be performed without affecting system functionality by using standardized data formats and structured storage practices. This ensures that the system can scale as needed and adapt to changing requirements.

The migration process can include transferring user data, quiz history, and real estate datasets to a new storage system while maintaining data integrity and consistency. This forward-compatible design ensures long-term sustainability of the application.

# 4. Interfaces

The Interfaces module defines how different parts of the system interact with each other as well as how users communicate with the application. In the Real Estate Quiz/Tutor Bot, interfaces play a critical role in ensuring seamless communication between the frontend, backend, and Generative AI components.

The primary interface is the **User Interface (UI)**, which acts as the interaction layer between the user and the system. It is designed to be simple, intuitive, and responsive, allowing users to easily start quizzes, view dynamically generated questions, submit answers, and receive feedback. The UI presents information in a structured manner, ensuring that users can clearly understand both the questions and the explanations provided by the system. A well-designed interface enhances user engagement and ensures a smooth learning experience.

The **Backend Interface** consists of RESTful APIs that handle all communication between the frontend and backend systems. These APIs are responsible for receiving user inputs, processing requests, and sending appropriate responses. Each API endpoint is designed with a specific purpose, such as initiating a quiz session, fetching questions, evaluating answers, and delivering feedback. The backend interface ensures that all operations are performed efficiently and securely while maintaining consistency in data exchange.

Another important interface is the **Generative AI Interface**, which connects the backend system with the Large Language Model (LLM). The backend sends structured prompts to the AI model and receives generated outputs such as quiz questions, evaluation results, and explanations. This interaction is carefully designed to ensure that the AI receives proper context and produces accurate, relevant, and meaningful responses.

Additionally, the system includes a **Storage Interface**, which allows the backend to interact with local storage for retrieving and storing data. This interface ensures efficient data access and management, supporting operations such as fetching real estate data, storing quiz results, and maintaining user progress.

Overall, the interface design ensures smooth communication across all components, enabling the system to function as a cohesive and efficient application.

## 5. State and Session Management

State and session management are essential components of the system, as they ensure continuity and consistency during user interactions. In the Real Estate Quiz/Tutor Bot, the system must track the progress of each user throughout the quiz session, including the current question, previous responses, scores, and overall performance.

When a user initiates a quiz, a unique session is created and assigned to that user. This session acts as a temporary container that stores all relevant information related to the user's interaction with the system. The session includes details such as the sequence of questions presented, answers submitted by the user, scores obtained, and feedback generated by the AI. This allows the system to maintain a continuous flow of interaction without losing context.

The system ensures that session data is managed efficiently to avoid memory overhead and performance issues. Session information is stored temporarily and is cleared once the quiz is completed or the session expires. This approach helps in optimizing resource utilization while maintaining a smooth user experience.

In addition to maintaining active sessions, the system is also capable of handling interruptions. For example, if a user leaves the quiz midway, the system can be designed to allow session recovery, enabling the user to resume from where they left off. This enhances usability and ensures a better user experience.

Proper state management also ensures that the system can handle multiple users simultaneously without conflicts. Each session is isolated, preventing data overlap and ensuring accurate tracking of individual user progress. Overall, effective session management is crucial for delivering a reliable and personalized learning experience.

## 6. Caching

Caching is an important performance optimization technique used in the system to improve response time and reduce computational overhead. In the Real Estate Quiz/Tutor Bot, caching plays a key role in minimizing repeated processing, especially when dealing with Generative AI operations, which can be computationally expensive.

The system can cache frequently accessed data such as commonly used real estate information, previously generated quiz questions, and standard explanations. By storing this data in a temporary memory layer, the system can quickly retrieve it when needed instead of recomputing or regenerating it using the AI model. This significantly reduces response time and enhances the overall user experience.

Caching is particularly useful in scenarios where multiple users request similar types of questions or information. Instead of sending repeated requests to the LLM, the system can serve cached responses, thereby reducing API usage and improving efficiency. This also helps in lowering operational costs associated with AI model usage.

The caching mechanism is designed to ensure that data remains consistent and up-to-date. Cached data is periodically refreshed or invalidated to reflect any updates in the underlying data. This ensures that users always receive accurate and relevant information.

Furthermore, caching contributes to system scalability by reducing the load on backend services and AI components. As the number of users increases, caching helps maintain system performance by minimizing redundant processing. Overall, an effective caching strategy enhances both performance and scalability while ensuring a smooth and responsive user experience.

# 7. Non-Functional Requirements

## 7.1 Security Aspects

Security is an important consideration in the system design to ensure safe handling of user data and system operations. The system implements validation mechanisms to prevent invalid or malicious inputs from affecting the application.

Access to APIs is controlled to ensure that only authorized users can interact with the system. Sensitive data is handled securely, and proper measures are taken to protect against common vulnerabilities such as data leakage and unauthorized access.

The system is designed to maintain data integrity and confidentiality, ensuring a secure and reliable user experience.

## 7.2 Performance Aspects

The system is designed to deliver high performance by ensuring fast response times and efficient handling of user requests. The use of caching mechanisms and optimized data access reduces latency and improves system responsiveness.

The architecture supports scalability, allowing the system to handle multiple users simultaneously without performance degradation. Efficient communication between components ensures smooth processing of requests and quick delivery of results.

Overall, the system is optimized to provide a responsive and seamless user experience while maintaining high efficiency.

# 8. References

1. FastAPI Documentation.
   https://fastapi.tiangolo.com/

2. Groq API Documentation.
   https://platform.groq.com/docs/

3. FAISS (Facebook AI Similarity Search) Documentation.
   https://faiss.ai/

4. Pinecone Local Database Documentation.
   https://www.pinecone.io/docs/

5. ChromaDB Documentation.
   https://docs.trychroma.com/

6. Google AI – Generative AI Resources.
   https://ai.google.dev/

7. Vaswani, A., et al. (2017). *Attention Is All You Need.*
   https://arxiv.org/abs/1706.03762

8. Brown, T. B., et al. (2020). *Language Models are Few-Shot Learners.*
   https://arxiv.org/abs/2005.14165

9. Fielding, R. T. (2000). *Architectural Styles and REST.*
   https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm